Faculty of Computer Applications &

Information Technology

Integrated Master of Science (IT) – *i*MSc(IT)

Programme

Semester IX

Project Report for
221601906 Natural Language Processing - Practicals

"ProjectMind AI"

Submitted by:

Group No: 34

Tirth Vaghela, 202201619010225
Sakshi Patel, 202201619010173

Table of Contents

1. Problem Statement **3**
1.1 Background 3
1.2 Problem Addressed 3
1.3 Limitations of Keyword Search 3
1.4 Aim 3
1.5 Objectives 4
1.6 Scope and Limitations 4

2. Dataset Description **5**
2.1 Dataset Source 5
2.2 Number of Samples 5
2.3 Data Format 6
2.4 Example Data 7
2.5 Validation and Pre-processing of the Dataset 8

3. System Architecture **9**
3.1 Components 10
3.2 System Flow Diagram 11

4. Data Processing Workflow **12**
4.1 Mapping of the Guideline Steps to the Implementation 14
4.2 Abbreviations and Stop Words 14
4.3 Implementation of the Sparse Path 15
4.4 Implementation of the Dense Path 15

5. Model Architecture & Implementation **16**
5.1 Software Requirements 16
5.2 Hardware Requirements 16
5.3 Libraries and Technologies 17
5.4 Model Architecture 18
5.5 Implementation Details 19
5.6 Implementation Screenshots 21

6. Result Visualization **28**
6.1 Evaluation Method 28
6.2 Metrics 28
6.3 Why a Confusion Matrix Is Not Used 28
6.4 Recorded Results 29
6.5 Reproducing the Evaluation 31
6.6 Automated Tests 32
6.7 Limitations 32

7. References **33**

1. Problem Statement

1.1 Background

Students in the Integrated Master of Science (IT) programme are expected to choose a semester or final-year project that matches their interests, their current skills and the time available. Project ideas are usually collected from websites, repositories, seniors and faculty suggestions. This information is spread across many sources, is described in inconsistent ways and rarely states the technologies, prerequisite skills or expected duration of a project.

1.2 Problem Addressed

ProjectMind AI addresses the task of finding suitable project ideas from a description written in plain English. A student should be able to type a request such as “I want an AI-based healthcare project using Python” and receive a ranked list of relevant projects, together with the technologies, difficulty, estimated duration, required skills and learning resources of each project.

1.3 Limitations of Keyword Search

Conventional search matches the exact words of a query against the words of a document [1]. This fails when the student and the project description use different words for the same idea; for example, a query about “healthcare AI” may not match a project described as a “clinical decision support system”. Pure meaning-based (dense embedding) search has the opposite weakness: it can rank a project that mentions a rare technical term such as “YOLOv8” below generic projects of the same domain, because the encoder does not separate such low-frequency tokens well. The source code of the project (backend/nlp/similarity.py) states both failure modes as the reason for combining the two approaches.

1.4 Aim

The aim of the project is to build a web application that recommends student projects from a natural-language description by combining semantic sentence embeddings [2] with BM25 keyword retrieval [3] over a structured project catalogue.

1.5 Objectives

1. To prepare a structured catalogue of project briefs with consistent fields for title, description, domain, technologies, difficulty, duration, skills, keywords and learning resources.
2. To implement a text preprocessing module that applies minimal cleaning for the transformer model and the classical pipeline (tokenisation, stop-word removal and lemmatisation) for keyword retrieval.
3. To represent every project and every query as a 384-dimensional sentence embedding using the pretrained all-MiniLM-L6-v2 model [4].
4. To rank projects by a weighted fusion of semantic similarity and BM25 relevance, with filters for domain, difficulty, technology and maximum duration.
5. To evaluate retrieval quality on a labelled set of queries using standard information-retrieval metrics.
6. To expose the system through a Flask REST API and a React user interface.

1.6 Scope and Limitations

The implemented scope covers natural-language search over a catalogue of 500 project briefs, filtering, similar-project lookup, explanation of matched terms, skill-gap analysis, generated synopses and roadmaps, a résumé matcher, an idea-originality check, team workspaces and an optional AI assistant. The NLP evaluation in this report concerns the hybrid retrieval engine only.

The catalogue contains project briefs rather than completed projects, and the github_url field links to a reference repository for the main technology. The sentence-embedding model is used as supplied and is not trained or fine-tuned on the catalogue. The AI assistant and AI idea generation depend on an external Groq API key and are not part of the retrieval pipeline. The search index is not rebuilt automatically when the catalogue changes.

2. Dataset Description

2.1 Dataset Source

The dataset is a catalogue of project briefs prepared for this application and stored in the repository as backend/dataset/projects_seed.json. It is not a public benchmark dataset and was not downloaded from an external source. The script scripts/seed_db.py loads the file into the projects collection of a MongoDB database, from which the search index is built.

A second file, backend/dataset/eval_queries.json, contains labelled queries used to evaluate retrieval quality (Section 6).

2.2 Number of Samples

One sample is one project brief. The file contains 500 records, each with a unique project_id from pm-001 to pm-500, distributed over 14 domains as shown in Table 1. The counts were obtained by reading the file.

Table 1: Distribution of Projects by Domain

Domain	Number of projects
Web Development	51
Natural Language Processing	46
Machine Learning	44
Computer Vision	43
Deep Learning	39
Data Science	39
Internet of Things	34
Cybersecurity	34
Mobile Development	33
Artificial Intelligence	31
Blockchain	30
Cloud & DevOps	27
Robotics	27
Augmented & Virtual Reality	22
Total	**500**

Every record also has a difficulty level and an estimated duration between 3 and 14 weeks. The difficulty distribution is given in Table 2.

Table 2: Distribution of Projects by Difficulty

Difficulty	Number of projects
Beginner	78
Intermediate	226
Advanced	196
Total	**500**

The evaluation file contains 25 labelled queries. Each query is paired with the list of project_ids judged relevant to it, an average of 16.1 relevant projects per query. The queries cover the 14 domains and include short requests ("IoT smart home automation"), technology-specific requests ("YOLOv8 object detection") and sentence-style requests ("I want a computer vision project using Python").

2.3 Data Format

The catalogue is a JSON array of objects. After seeding, each object is stored unchanged as a document in the MongoDB projects collection. The fields are described in Table 3.

Table 3: Dataset Fields and Descriptions

Field	Type	Description
project_id	String	Unique identifier, e.g. pm-001
title	String	Project name
description	String	Short summary of what the project does
domain	String	One of the 14 allowed domains
technologies	List of strings	Technologies used, e.g. Python, Flask
difficulty	String	Beginner, Intermediate or Advanced
duration_weeks	Integer	Estimated completion time in weeks
skills	List of strings	Skills required for the project
keywords	List of strings	Search tags for the project
github_url	String	Reference repository for the main technology
learning_resources	List of strings	Tutorial and documentation links

2.4 Example Data

The first record of the catalogue is reproduced below without changes.

Source: backend/dataset/projects_seed.json (record pm-001)

```
{
  "project_id": "pm-001",
  "title": "AI-Powered Medical Diagnosis Assistant",
  "description": "A clinical decision support system that predicts
    likely conditions from patient-reported symptoms, vitals and basic
    lab values. Trains a gradient boosting classifier on an open medical
    dataset, exposes predictions through a REST API, and shows
    per-feature contributions so a clinician can see why a prediction
    was made rather than trusting a black box.",
  "domain": "Artificial Intelligence",
  "technologies": ["Python", "scikit-learn", "Flask", "Pandas", "SHAP"],
  "difficulty": "Advanced",
  "duration_weeks": 12,
  "skills": ["Supervised learning", "Feature engineering",
    "Model interpretability", "REST API development",
    "Medical data handling"],
  "keywords": ["healthcare", "diagnosis", "classification",
    "explainable ai", "clinical"],
  "github_url": "https://github.com/shap/shap",
  "learning_resources": ["https://scikit-learn.org/stable/user_guide.html",
    "https://shap.readthedocs.io/"]
}
```

The description is wrapped over several lines here for printing. The title, description, technologies, skills and domain are the fields used to build the text that is embedded and indexed (Section 5.5). The keywords field is stored with the record but is not part of the indexed text. The difficulty, domain, technologies and duration_weeks fields are used as search filters.

2.5 Validation and Pre-processing of the Dataset

The function validate() in backend/db/schema.py checks every record before use. It reports a record whose domain is not one of the 14 allowed domains, whose difficulty is not Beginner, Intermediate or Advanced, or whose duration_weeks lies outside the range 1 to 52. The script scripts/validate_dataset.py applies these checks to the whole catalogue.

Near-duplicate projects are detected by HybridRetriever.find_duplicates() in backend/nlp/similarity.py, which reports every pair of projects whose embeddings have a cosine similarity [1] of at least 0.92 (DUPLICATE_THRESHOLD in config.py).

No missing-value imputation is performed; a record that fails validation is reported rather than repaired. The catalogue has no train/test split because no model is trained on it: the sentence-embedding model is used as pretrained, and BM25 builds its term statistics directly from the catalogue text. The labelled queries in eval_queries.json are used only for evaluation.

3. System Architecture

ProjectMind AI is a client–server web application with three layers: a React single-page application in the browser, a Flask REST API, and a data layer made of a MongoDB database and an on-disk vector index. Figure 1 shows the components identified in the source code.

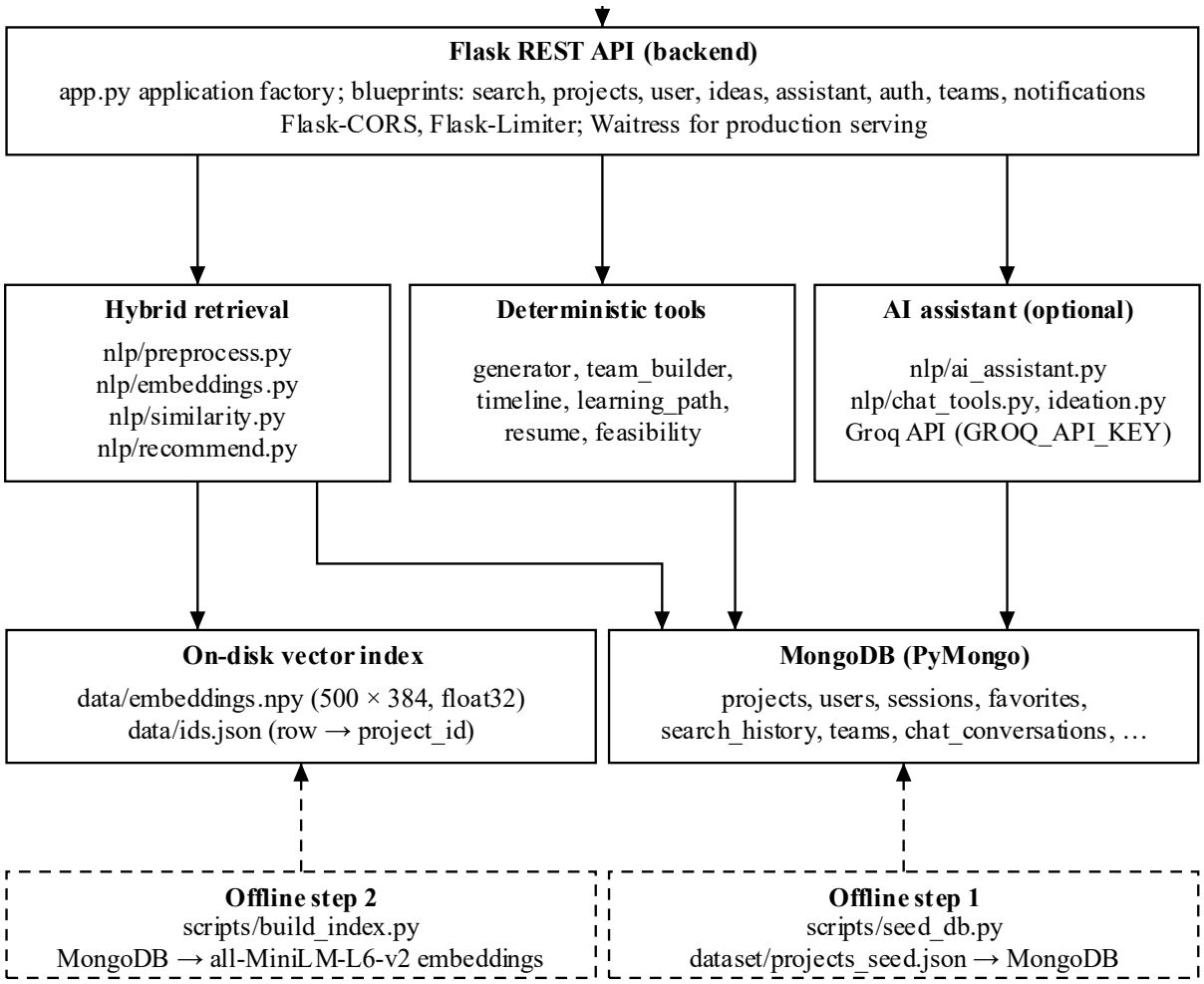


Figure 1: System Architecture of ProjectMind AI

Dashed boxes are offline steps that are run by the operator before the server starts, not on each request.

3.1 Components

- **Frontend.** A React 19 application [5] built with Vite and styled with Tailwind CSS. It calls the backend through axios (frontend/src/services/api.js) and sends an anonymous X-Device-Id header, or an X-Auth-Token header for signed-in users.
- **Flask REST API.** backend/app.py creates the application and registers eight blueprints [6]. Every response is returned as {"data": …} on success or {"error": …} on failure. Flask-Limiter applies rate limits to the routes that are expensive to serve; for example, POST /api/search is limited to 30 requests per minute.
- **Hybrid retrieval.** preprocess.py prepares text, embeddings.py loads the sentence-transformer model and the vector index, similarity.py contains the HybridRetriever class that scores and ranks projects, and recommend.py builds the shared retriever and adds explanations and personalisation.
- **Deterministic tools.** Skill-gap analysis, synopsis and roadmap generation, team-role suggestions, timeline prediction, learning paths and résumé parsing are rule-based functions over the project fields. They do not call an external service.
- **AI assistant (optional).** The project chat, the Dashboard assistant and AI idea generation call the Groq API. They are enabled only when GROQ_API_KEY is set and are not used by the search pipeline.
- **Data layer.** MongoDB [7] stores projects, favourites, search history, feedback, conversations, users, sessions, teams, team messages and notifications. The project embeddings are stored separately as data/embeddings.npy, with data/ids.json mapping each matrix row to a project_id.

3.2 System Flow Diagram

Figure 2 shows the processing of one search request, from the query typed by the user to the ranked result cards. The steps follow the code of routes/search.py and HybridRetriever.search().

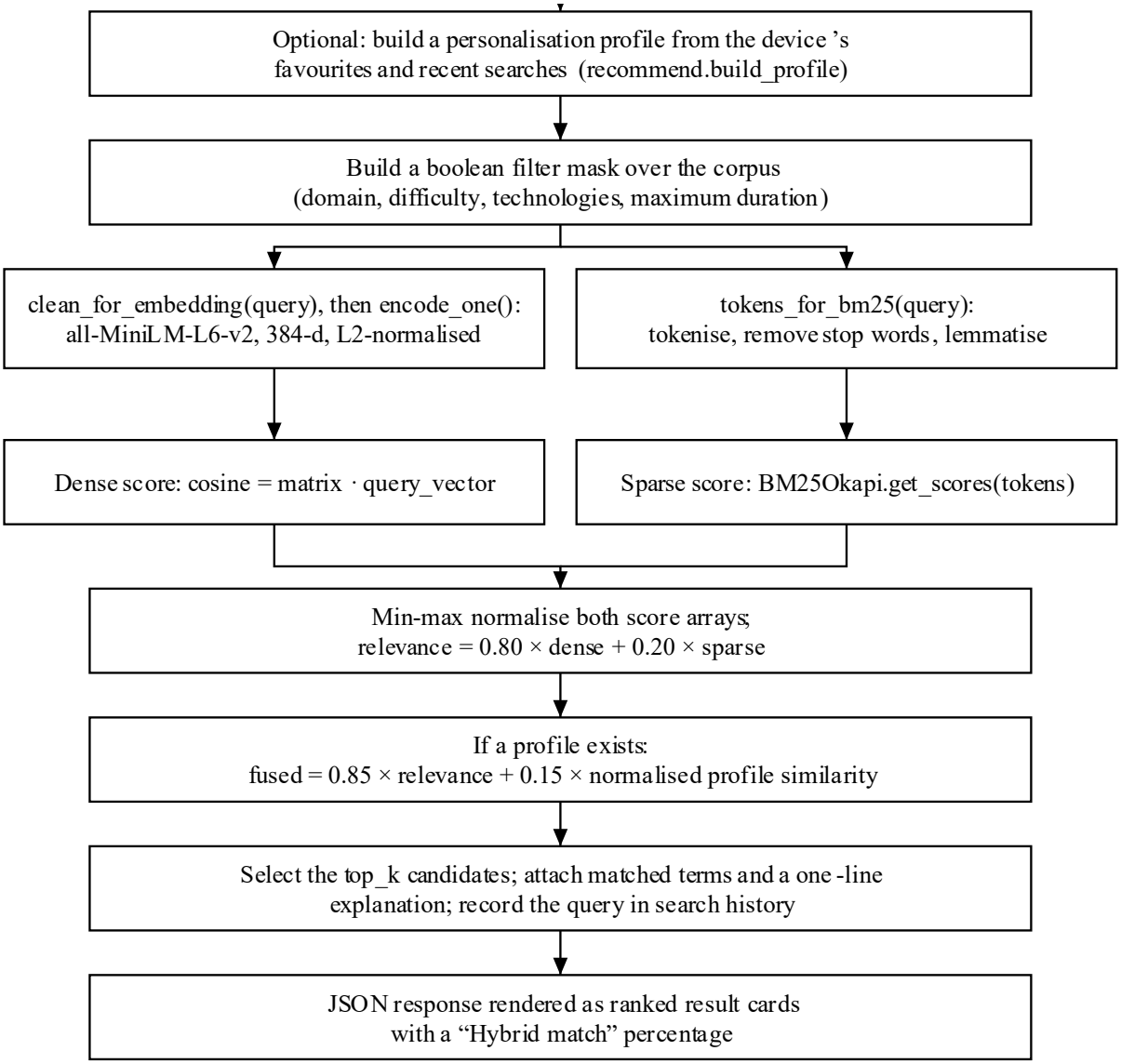


Figure 2: System Flow of a Search Request

The filter mask is built before any scores are calculated, so only projects that satisfy every filter are ranked. The source code explains that ranking first and filtering the top results afterwards can return no results, because the requested domain or difficulty may not appear among the top ten unfiltered matches.

The dense and sparse scores are calculated for the same set of candidate projects and then combined. Personalisation is applied only when the device has saved projects or a search history and the user has not switched it off; its weight is limited to 15% of the final score. The query is recorded in the search history after the results have been computed, and a failure to record it does not affect the response.

4. Data Processing Workflow

The university guidelines describe the sequence Raw text → Tokenization → Stop-word removal → Stemming/Lemmatization → Vectorization. ProjectMind AI implements these steps, but not as a single chain. backend/nlp/preprocess.py defines two separate paths over the same text, shown in Figure 3 (next page).

- **Dense path** (clean_for_embedding): minimal cleaning only, because the sentence-transformer model was trained on natural sentences [2] and uses word order and function words to build its representation.
- **Sparse path** (tokens_for_bm25): the full classical pipeline, because BM25 is a bag-of-words model that benefits from removing stop words and merging inflected forms [1].

The module documentation gives an example of why stop words are kept on the dense path: “a project for kids with disabilities” and “a project by kids with disabilities” become the same set of tokens once stop words are removed, and the model could no longer distinguish them.

(b) Implementation in nlp/preprocess.py: two separate paths over the same text

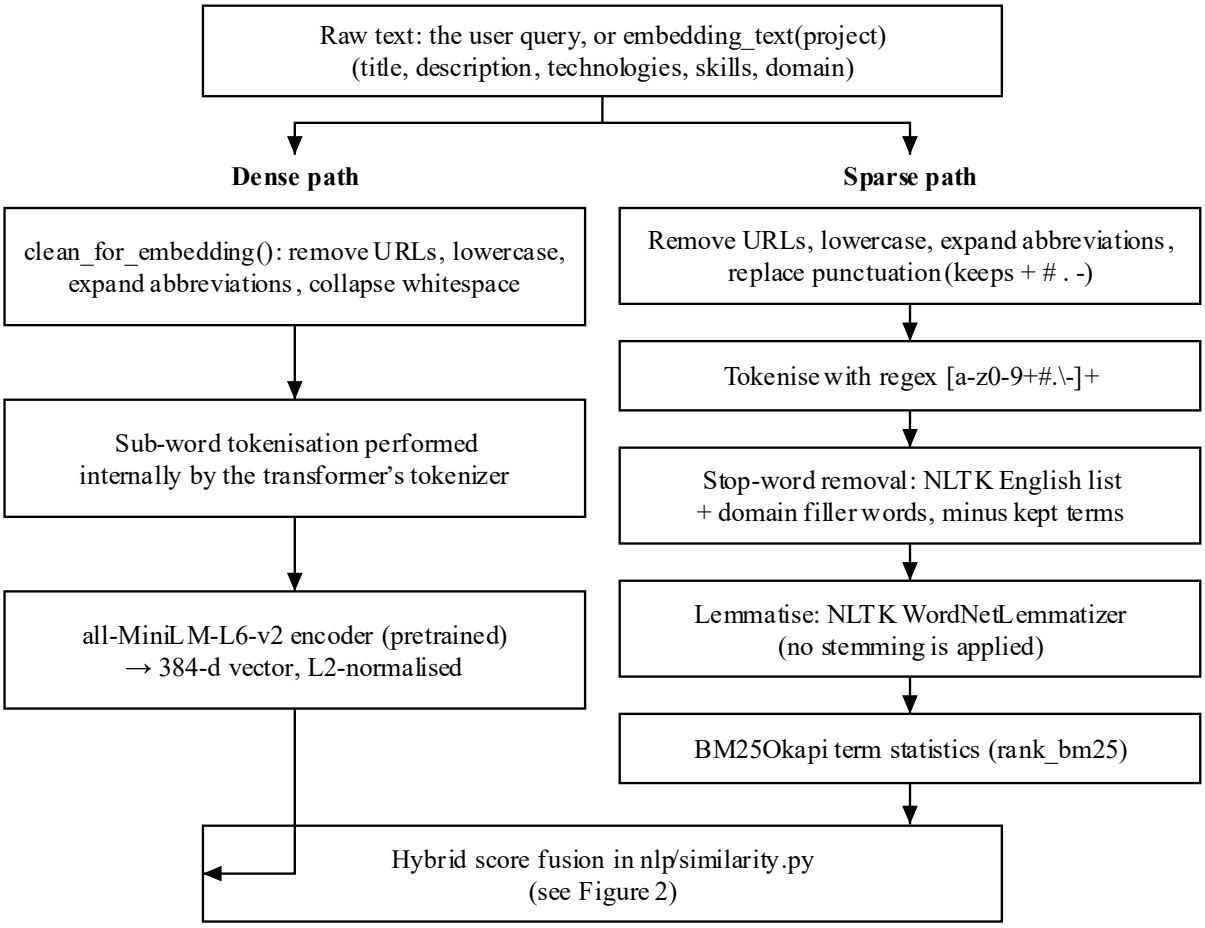


Figure 3: NLP Data Processing Workflow

Both paths start from the same raw text. For a project, the raw text is produced by embedding_text() from the title, description, technologies, skills and domain; for a search, it is the query typed by the user.

4.1 Mapping of the Guideline Steps to the Implementation

Table 4 compares each step of the sequence in the guidelines with the corresponding code.

Table 4: Guideline Steps and Their Implementation

Step	Dense path	Sparse (BM25) path
Raw text	Query, or embedding_text(project)	Same text as the dense path
Cleaning	Remove URLs, lowercase, expand abbreviations, collapse whitespace	Remove URLs, lowercase, expand abbreviations, replace punctuation except + # . -
Tokenization	Performed inside the model by its own sub-word tokenizer	Regular expression [a-z0-9+#.\-]+, then leading and trailing "." and "-" removed
Stop-word removal	Not applied	NLTK English stop-word list with domain changes (Section 4.2)
Stemming / Lemmatization	Not applied	NLTK WordNetLemmatizer; stemming is not used
Vectorization	all-MiniLM-L6-v2 sentence embedding, 384 dimensions, L2-normalised	BM25Okapi index built from the token lists

4.2 Abbreviations and Stop Words

Both paths expand 15 standalone abbreviations that students commonly type, for example "ml" → "machine learning", "nlp" → "natural language processing", "cv" → "computer vision" and "iot" → "internet of things". The expansion matches whole words only, so abbreviations inside other words are left unchanged.

The stop-word list for the sparse path starts from the NLTK English list [8] and is changed in two ways. The words "c", "r", "computer", "system", "web", "data" and "network" are removed from the list because they are meaningful in this domain; "c" and "r" are also exempt from the rule that drops one-character tokens. Words that appear in almost every query and project, such as "project", "idea", "want", "build", "using", "final", "year" and "student", are added. The code comment records that, before this change, the query "beginner project for farming" returned an MNIST digit-recognition project at rank 3 only because of the token "project".

4.3 Implementation of the Sparse Path

The function below performs the cleaning, tokenisation, stop-word removal and lemmatisation steps of the sparse path. It is reproduced from the source file without changes.

Source: backend/nlp/preprocess.py

```
def tokens_for_bm25(text: str) -> list[str]:
    """Full classical pipeline: clean -> tokenise -> stopwords -> lemmatise.
    ..."""
    if not text:
        return []

    text = _URL_RE.sub(" ", text.lower())
    text = expand_abbreviations(text)
    text = _PUNCT_RE.sub(" ", text)

    stop, lemmatizer = _nltk_tools()

    tokens: list[str] = []
    for token in _TOKEN_RE.findall(text):
        token = token.strip(".-")
        if token in stop:
            continue
        if len(token) < 2 and token not in _SHORT_MEANINGFUL_TOKENS:
            continue
        tokens.append(lemmatizer.lemmatize(token))

    return tokens
```

The stop-word set and the lemmatiser are loaded once by _nltk_tools(), which downloads the NLTK stopwords, wordnet and omw-1.4 resources if they are missing and caches the result. The regular expressions keep the characters +, #, . and - so that technology names such as “C++”, “C#”, “Node.js” and “scikit-learn” survive cleaning.

4.4 Implementation of the Dense Path

Source: backend/nlp/preprocess.py

```
def clean_for_embedding(text: str) -> str:
    if not text:
        return ""
    text = _URL_RE.sub(" ", text)
    text = text.lower()
    text = expand_abbreviations(text)
    return _WHITESPACE_RE.sub(" ", text).strip()
```

The docstring of the function is omitted here. Word order, stop words and inflections are left unchanged; the cleaned text is passed to the sentence-transformer model, which tokenises it internally.

5. Model Architecture & Implementation

5.1 Software Requirements

Table 5 lists the software used by the project. Backend versions are the exact versions in backend/requirements.lock.txt; frontend versions are the minimum versions declared in frontend/package.json.

Table 5: Software Requirements

Software	Version	Purpose
Python	3.11	Backend runtime
Flask	3.1.3	REST API framework
MongoDB (PyMongo driver)	PyMongo 4.17.0	Database and driver
PyTorch (CPU build)	2.13.0+cpu	Runs the transformer model
Node.js and npm	Not specified	Frontend build tooling
React	^19.2.8	User interface
Vite	^8.2.0	Development server and build
Tailwind CSS	^4.3.3	Styling
pytest	~8.3	Backend tests
Vitest	^4.1.11	Frontend tests

The MongoDB server version is not stated in the repository. The README specifies a local mongod instance for development and MongoDB Atlas for production; deployment of the frontend to Vercel and of the backend to Render is also described there.

5.2 Hardware Requirements

The project installs the CPU-only build of PyTorch, and the README explains that this avoids downloading the CUDA build of about 2.5 GB, which the project does not use. No GPU is required. The embedding model is about 90 MB and is downloaded on first use, and the index of 500 vectors of 384 float32 values occupies about 750 KB. The configuration of the computer used for development is not recorded in the repository and is therefore not stated here.

5.3 Libraries and Technologies

Table 6: Libraries Used by the NLP and Backend Modules

Library	Version	Role in the project
sentence-transformers	3.4.1	Loads all-MiniLM-L6-v2 and encodes text into normalised 384-d vectors
transformers	4.57.6	Underlying transformer implementation used by sentence-transformers
rank-bm25	0.2.2	BM25Okapi keyword index and scoring
NLTK	3.10.3	English stop-word list and WordNet lemmatiser
NumPy	2.5.2	Embedding matrix, dot products, normalisation and top-k selection
Flask-CORS	5.0.1	Cross-origin access for the frontend
Flask-Limiter	3.12	Rate limiting of costly routes
python-dotenv	~1.0	Loads configuration from backend/.env
pypdf / python-docx	5.9.0 / 1.2.0	Extracts text from uploaded résumés
groq	0.37.1	Optional AI assistant (not used for search)
Waitress	3.0.2	Production WSGI server

spaCy (3.8.16) and scikit-learn (1.9.0) are listed in requirements.txt, and the README describes scikit-learn as providing BM25-style scoring. A search of the backend source found no import of either library: BM25 is computed by rank-bm25 and cosine similarity by NumPy. The two libraries are therefore installed but not used by the code examined for this report.

On the frontend, React Router handles page routing, axios performs HTTP requests, Framer Motion provides animation, Three.js draws the landing-page background and jsPDF exports synopses and roadmaps as PDF files.

5.4 Model Architecture

The retrieval model has no trainable parameters of its own. It combines a pretrained sentence encoder, a BM25 keyword model and a weighted score fusion.

Sentence encoder. sentence-transformers/all-MiniLM-L6-v2 is a six-layer MiniLM transformer fine-tuned by the Sentence-Transformers authors to map sentences and short paragraphs to 384-dimensional vectors for semantic search and clustering [2], [4], [9]. ProjectMind AI uses the published model without fine-tuning. Each vector is L2-normalised when it is encoded, so the cosine similarity [1] of a query vector q and a project vector d is the dot product q · d.

Keyword model. BM25 [3] scores a project for a query by summing, over the query terms, the inverse document frequency of each term multiplied by a saturating function of how often the term occurs in the project, adjusted for the length of the project text. The project uses the BM25Okapi class of rank-bm25 [10] with its default parameters (k1 = 1.5, b = 0.75).

Score fusion. The two kinds of score have different ranges: cosine similarity lies between −1 and 1, while BM25 is unbounded. Each array of candidate scores is therefore rescaled to the range 0 to 1 by min-max normalisation, and the final relevance is

$$relevance = 0.80 \times minmax(\text{dense}) + 0.20 \times minmax(\text{BM25})$$

A weighted linear combination of a sparse and a dense score has also been used in published dense-retrieval work [11]. The weights are DENSE_WEIGHT and SPARSE_WEIGHT in config.py; they can be changed through environment variables. If every candidate receives the same score from one signal, its normalised array is set to zero, so that signal does not affect the ranking.

Personalisation. When a device has saved projects or recent searches, recommend.build_profile() builds a unit “taste” vector from them, with recent searches given 0.4 of the weight of saved projects. The final score is then a convex combination:

$$fused = 0.85 \times relevance + 0.15 \times minmax(\text{profile} \cdot d)$$

This keeps the score between 0 and 1, so the interface can show it as a match percentage, and limits the effect of personalisation to 15% (PERSONALIZATION_WEIGHT).

Ranking. The top k results are selected with numpy.argpartition and then sorted. Each result carries its fused score, its raw semantic and keyword scores and the query terms it matched. The Search page requests 12 results; the API default is 10 and the maximum is 50.

5.5 Implementation Details

Building the index offline. scripts/build_index.py reads all projects from MongoDB in project_id order, builds one text per project and encodes them in batches of 32. The text is produced by the function below.

Source: backend/db/schema.py

```
def embedding_text(doc: dict[str, Any]) -> str:
    ...
    parts = [
        doc.get("title", ""),
        doc.get("description", ""),
        "Technologies: " + ", ".join(doc.get("technologies", [])),
        "Skills: " + ", ".join(doc.get("skills", [])),
        "Domain: " + doc.get("domain", ""),
    ]
    return ". ".join(part for part in parts if part).strip()
```

Encoding. The function below encodes a list of texts into normalised vectors. The model itself is loaded once and cached by get_model().

Source: backend/nlp/embeddings.py

```
def encode(texts: Iterable[str], show_progress: bool = False) -> np.ndarray:
    texts = list(texts)
    if not texts:
        return np.zeros((0, config.EMBEDDING_DIM), dtype=np.float32)

    vectors = get_model().encode(
        texts,
        batch_size=32,
        show_progress_bar=show_progress,
        convert_to_numpy=True,
        normalize_embeddings=True,
    )
    return np.asarray(vectors, dtype=np.float32)
```

save_index() writes the matrix to data/embeddings.npy and the list of ids to data/ids.json, and refuses to save if the number of vectors and ids differ. At request time only the query is encoded; cosine_scores() computes matrix @ query_vector for all projects in one matrix multiplication.

Hybrid scoring. The following lines of HybridRetriever.search() calculate the two scores for the filtered candidates and combine them.

Source: backend/nlp/similarity.py

```
clean_query = preprocess.clean_for_embedding(query)
query_tokens = preprocess.tokens_for_bm25(query)

# Dense half.
query_vector = embeddings.encode_one(clean_query)
dense_raw = embeddings.cosine_scores(query_vector, self.matrix)[candidates]

# Sparse half.
if self._bm25 is not None and query_tokens:
    sparse_raw = np.asarray(self._bm25.get_scores(query_tokens))[candidates]
else:
    sparse_raw = np.zeros(candidates.size, dtype=np.float32)

relevance = (
    config.DENSE_WEIGHT * _minmax(dense_raw)
    + config.SPARSE_WEIGHT * _minmax(sparse_raw)
)
```

Normalisation. The helper that rescales each score array:

Source: backend/nlp/similarity.py

```
def _minmax(scores: np.ndarray) -> np.ndarray:
    if scores.size == 0:
        return scores

    lo = float(scores.min())
    hi = float(scores.max())
    if hi - lo < 1e-9:
        return np.zeros_like(scores)

    return (scores - lo) / (hi - lo)
```

Final ranking. After the optional personalisation blend, the top k candidates are selected and sorted by score:

Source: backend/nlp/similarity.py

```
top_k = max(1, min(top_k, config.MAX_TOP_K, candidates.size))
top_local = np.argpartition(-fused, top_k - 1)[:top_k]
top_local = top_local[np.argsort(-fused[top_local])]
```

For each selected project, the matched terms are the intersection of the query tokens and the project tokens. recommend.explain() turns them into the one-line explanation shown on each result card.

Similar projects. HybridRetriever.similar() ranks projects by pure dense similarity to another project, excluding the project itself. BM25 is not used here because two full project descriptions are being compared rather than a short query and a description.

API integration. POST /api/search in backend/routes/search.py rejects an empty query or a query longer than 500 characters with HTTP 400, parses the filters and top_k, optionally builds the personalisation profile, calls recommend.search() and returns the ranked list together with the applied weights and personalisation details. A failure while building the profile or recording the history is logged, and the user still receives the search results.

Other features using the same retriever. The idea-originality check (POST /api/uniqueness) embeds a proposed idea and reports catalogue projects with a cosine similarity of at least 0.78 (UNIQUENESS_THRESHOLD). The résumé matcher (POST /api/resume-suggest) extracts text from a PDF, DOCX or TXT file, detects known technologies, skills and domains, and ranks projects with the same retriever, combined with skill fit.

5.6 Implementation Screenshots

Figures 4 to 14 are screenshots of the running application supplied with the source code. All were captured while signed out. They therefore show the search results and one project page with data, and the remaining pages in their empty or signed-out state; they do not demonstrate the signed-in features (teams, AI chat history) with data.

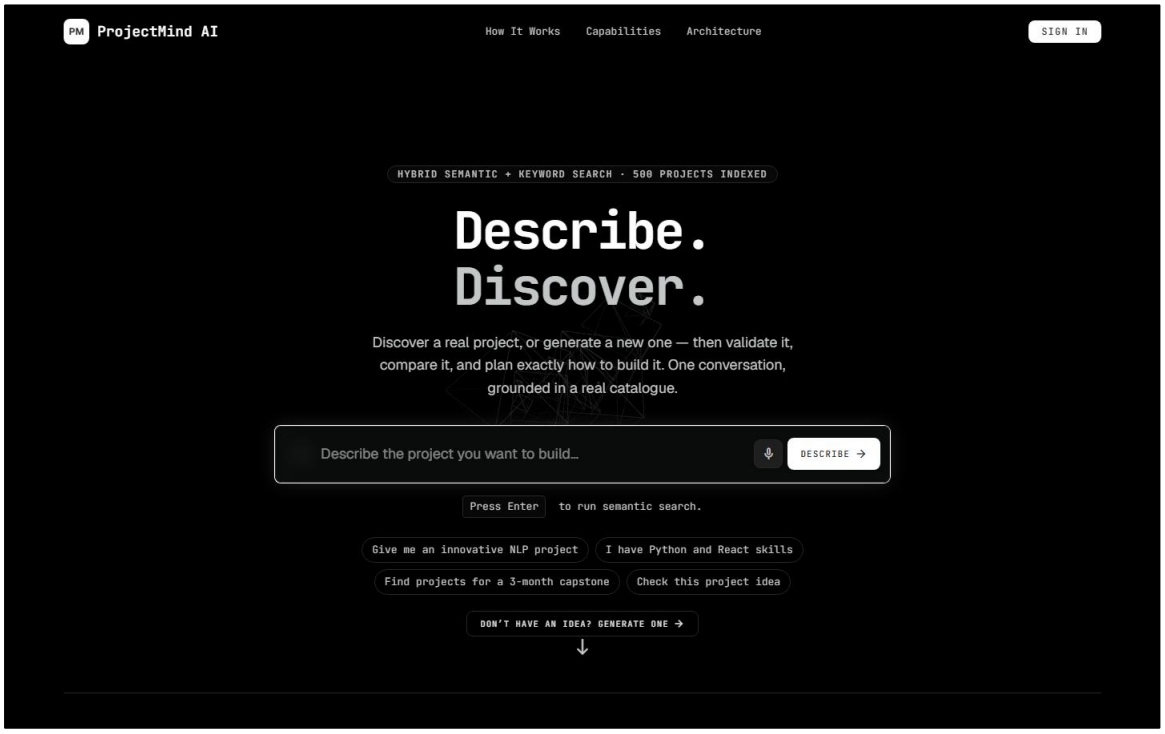


Figure 4: Landing page with the natural-language search box

The landing page states “500 projects indexed” and offers example queries.

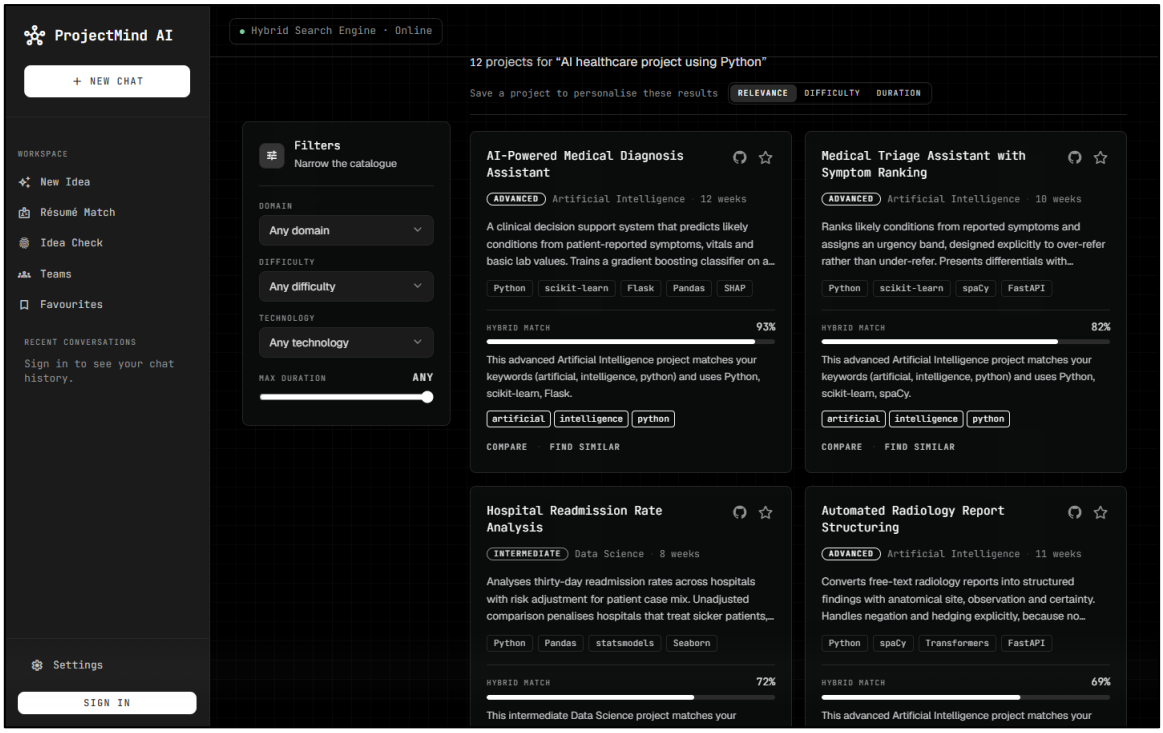


Figure 5: Hybrid search results for "AI healthcare project using Python"

Twelve results with filters, a hybrid match percentage and the matched terms (artificial, intelligence, python) for each card.

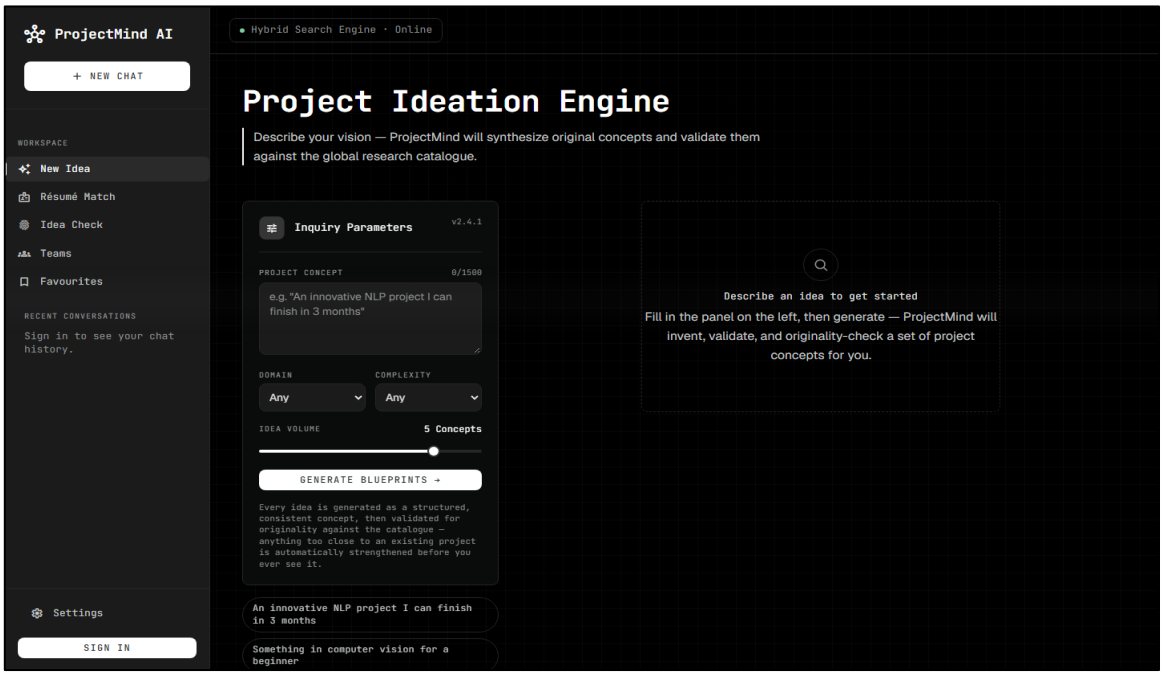


Figure 6: Project Ideation Engine (AI idea generation)

The form before any idea has been generated. This feature uses the optional Groq API.

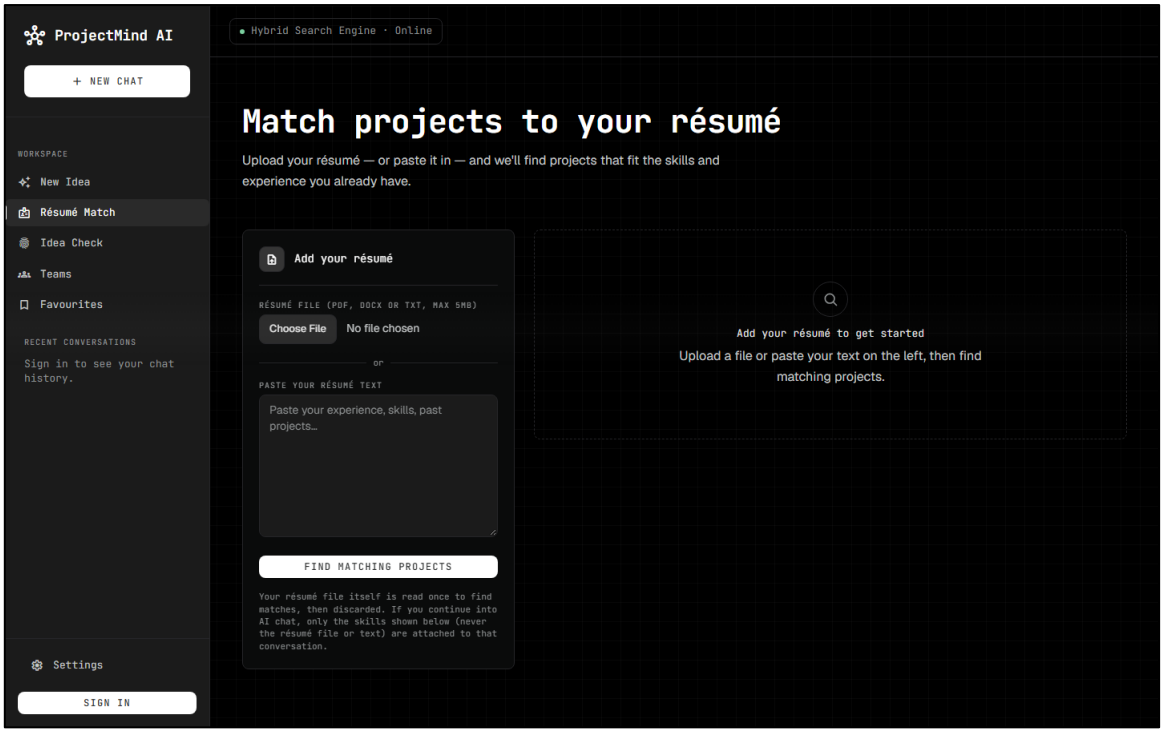


Figure 7: Résumé matcher

The upload and paste form before a résumé has been supplied.

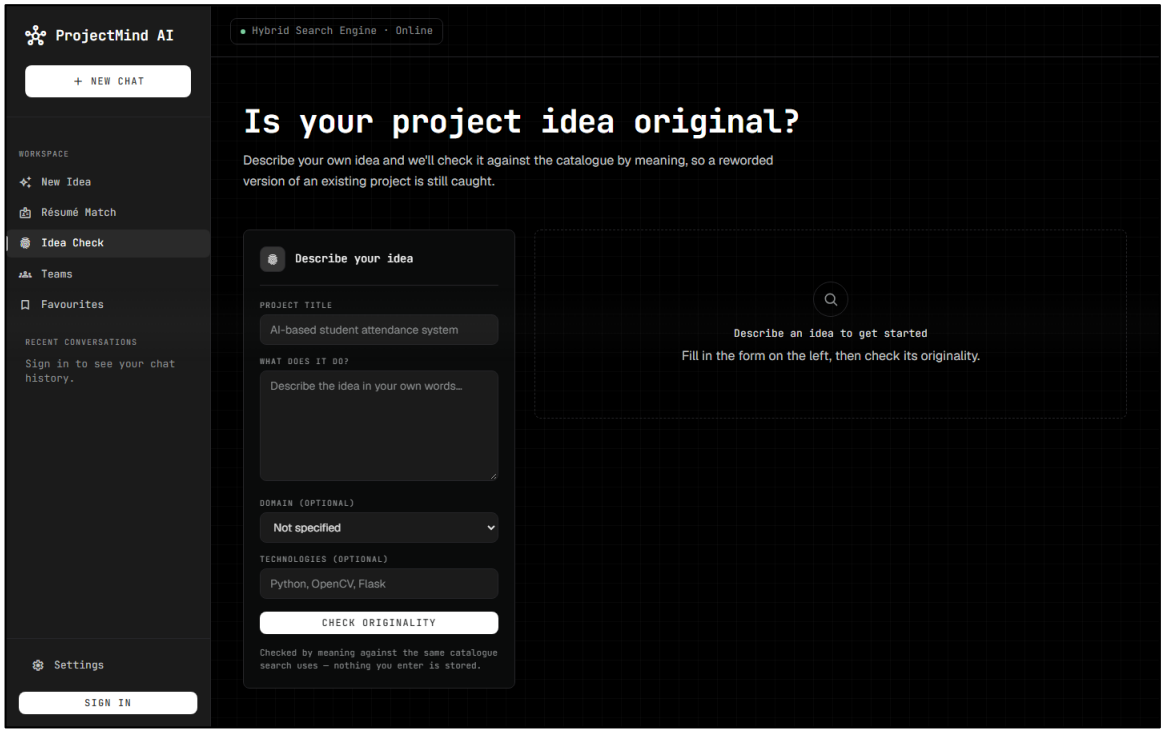


Figure 8: Idea originality check

The form for checking a proposed idea against the catalogue, before submission.

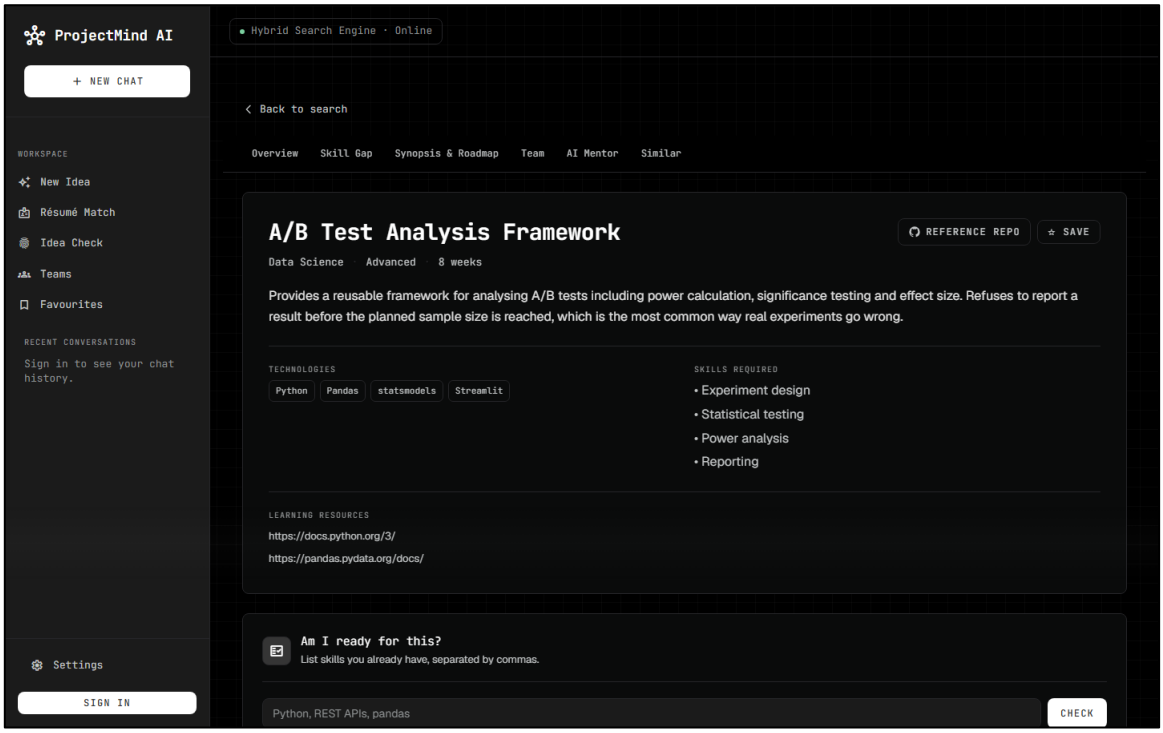


Figure 9: Project detail page

The record for “A/B Test Analysis Framework”, with technologies, required skills and learning resources, and the skill-readiness check.

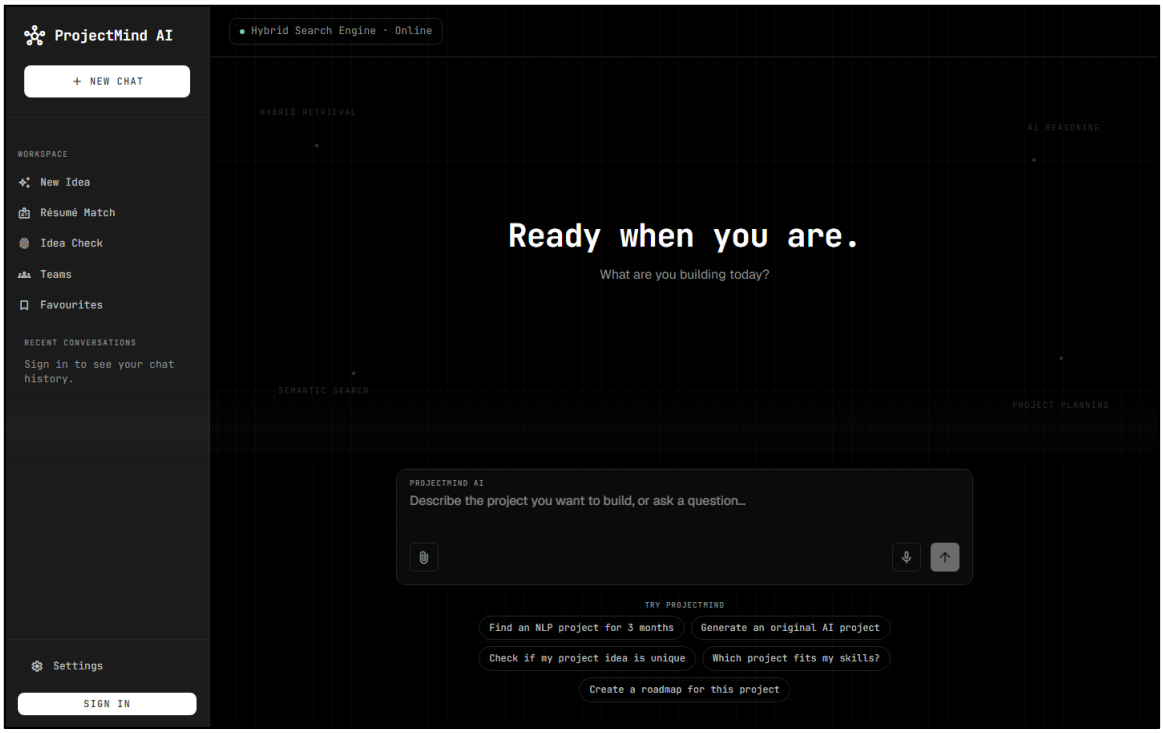


Figure 10: AI Dashboard

The conversational assistant in its empty state; chat history requires signing in.

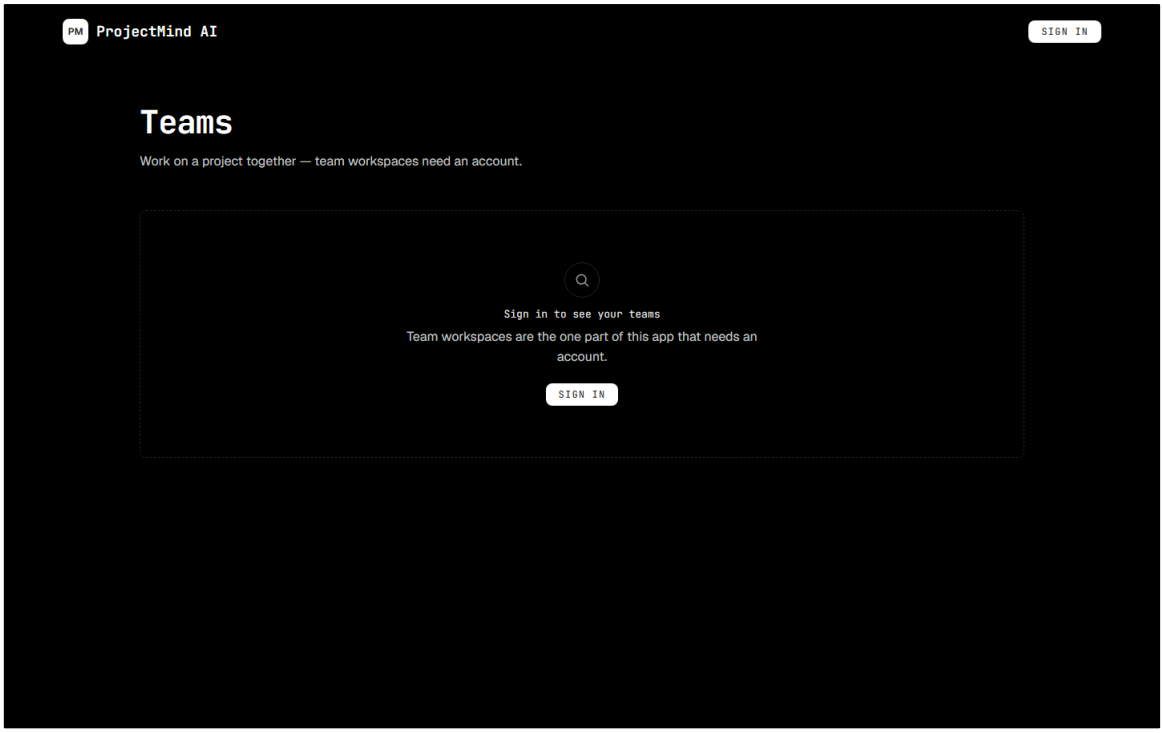


Figure 11: Teams page while signed out

Team workspaces require an account; the page asks the user to sign in.

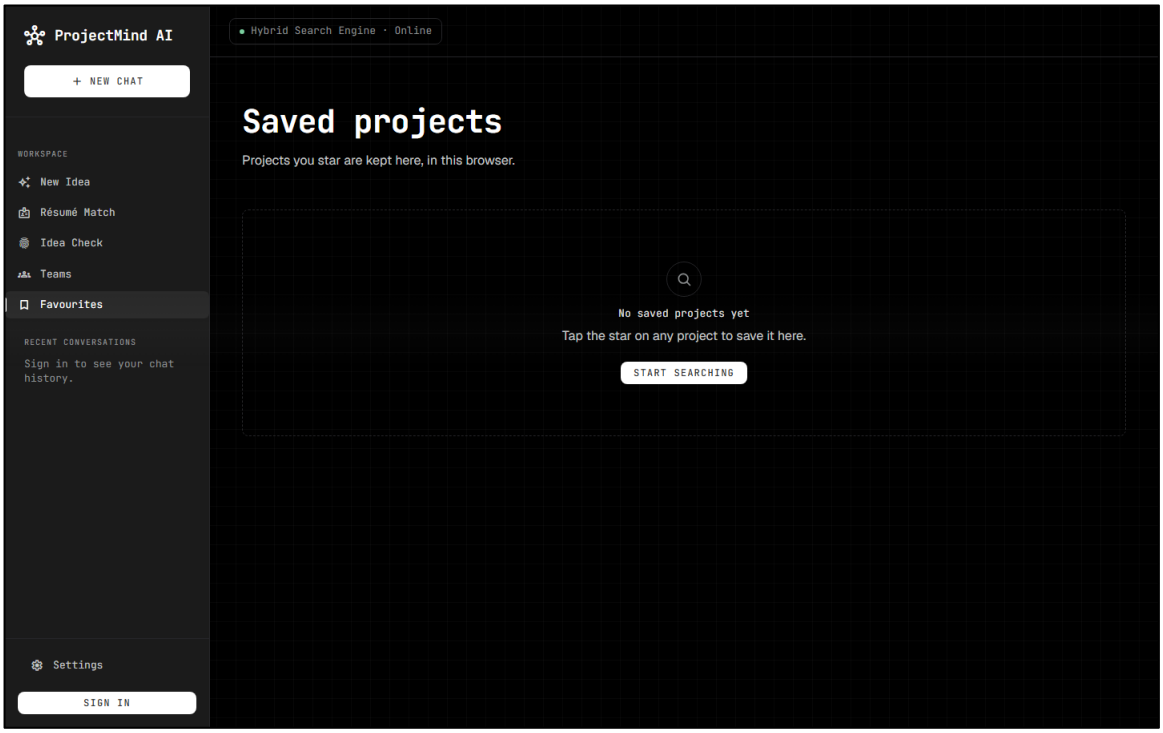


Figure 12: Favourites page

No projects have been saved on this device, so the list is empty.

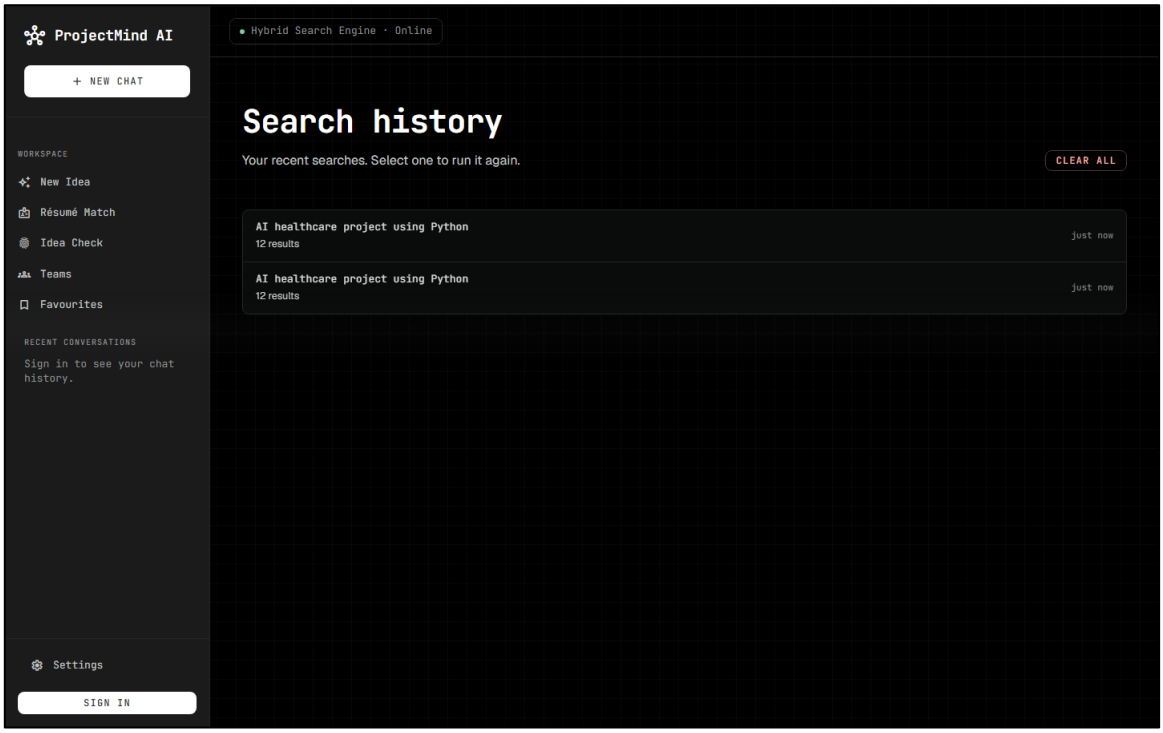


Figure 13: Search history

The device's recent searches, here two runs of the same query.

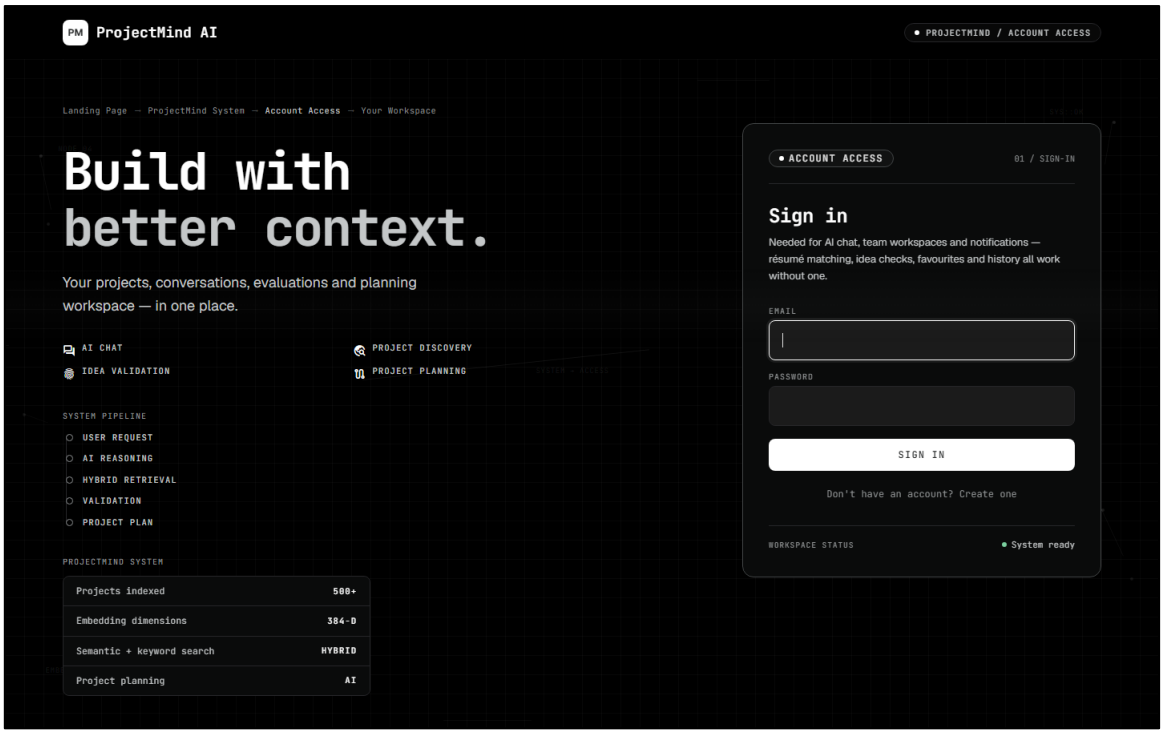


Figure 14: Account sign-in page

Sign-in form used for team workspaces and notifications.

6. Result Visualization

6.1 Evaluation Method

The retrieval engine is a ranking system, so it is evaluated with ranking metrics rather than classification metrics [1]. The script backend/scripts/evaluate.py runs each of the 25 labelled queries in eval_queries.json through HybridRetriever.search() with top_k = 10 and compares the ranked project_ids with the relevant ids listed for that query. Relevance is binary: a project is either listed as relevant or not.

6.2 Metrics

The script implements three metrics, each averaged over the 25 queries, with a default cut-off of k = 5:

- **Recall@k**: the number of relevant projects in the top k results divided by the total number of relevant projects for the query [1].
- **Mean Reciprocal Rank (MRR)**: 1 divided by the rank of the first relevant result, or 0 if no relevant result is returned in the top 10 [12].
- **nDCG@k**: the discounted cumulative gain of the top k results, in which a relevant result at position i contributes $1 / \log_2(i + 1)$, divided by the gain of an ideal ranking [13].

Because each query has 16.1 relevant projects on average, Recall@5 cannot reach 1 for most queries; nDCG@5 and MRR are more informative about the quality of the first results shown on screen.

The option --sweep repeats the evaluation for dense weights of 0.00, 0.20, 0.35, 0.50, 0.65, 0.80 and 1.00, with the sparse weight set to one minus the dense weight. This comparison is how the configured weights were chosen.

6.3 Why a Confusion Matrix Is Not Used

The guidelines suggest a confusion matrix, an accuracy graph or a performance comparison. ProjectMind AI does not classify text into categories, so it has no predicted and actual class labels from which a confusion matrix or classification accuracy could be computed. This report therefore presents a performance comparison of retrieval configurations.

6.4 Recorded Results

The results in this section were produced by running python -m scripts.evaluate --dataset v2 --sweep-json reports/eval_v2_sweep.json over the current catalogue of 868 projects and the 25 labelled queries of the version 2 dataset (eval_queries_v2.json), at a cut-off of k = 5. A project counts as relevant when its grade is 1 or 2. Each metric is calculated per query and macro-averaged over the 25 queries. In addition to the metrics of Section 6.2, Precision@5 is the number of relevant projects in the top five divided by five, and F1@5 is the harmonic mean of Precision@5 and Recall@5 for each query. All values in Table 7 and Figures 15 and 16 are taken from backend/reports/eval_v2_sweep.json and rounded to three decimal places.

Table 7: Evaluation Results for the 868-Project Catalogue

Dense weight	Sparse weight	Precision @5	Recall@ 5	F1@5	MRR	nDCG@ 5	Configuration
0.00	1.00	0.768	0.296	0.391	0.917	0.801	BM25 only
0.20	0.80	0.808	0.311	0.412	0.940	0.839	Hybrid
0.35	0.65	0.848	0.326	0.435	0.973	0.877	Hybrid
0.50	0.50	0.864	0.325	0.439	1.000	0.905	Hybrid
0.65	0.35	0.896	0.340	0.459	1.000	0.930	Hybrid
0.80	**0.20**	**0.920**	**0.356**	**0.478**	**1.000**	**0.945**	**Hybrid (configured, production)**
1.00	0.00	0.944	0.362	0.488	1.000	0.961	Dense only

The configured production weighting (DENSE_WEIGHT = 0.80, SPARSE_WEIGHT = 0.20 in backend/config.py) is shown in bold. The technology weight was 0.25 in every row.

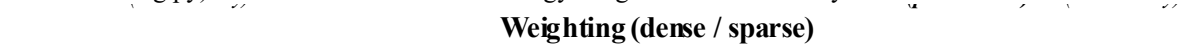


Figure 15: F1@5 Across Dense and Sparse Weight Configurations (source: eval_v2_sweep.json)



Figure 16: nDCG@5 Across Dense and Sparse Weight Configurations (source: eval_v2_sweep.json)

On these 25 queries, Precision@5, F1@5 and nDCG@5 rise steadily as the dense weight increases from 0.00 to 1.00, and MRR does not fall. The only exception is Recall@5, which is 0.326 at 0.35 / 0.65 and 0.325 at 0.50 / 0.50. BM25 alone gave the lowest value on every metric (F1@5 0.391, nDCG@5 0.801). Dense-only retrieval (1.00 / 0.00) gave the highest measured F1@5 (0.488) and nDCG@5 (0.961). The configured production weighting (0.80 / 0.20) gave an F1@5 of 0.478 and an nDCG@5 of 0.945, which is 0.010 below dense-only retrieval on

F1@5 and 0.016 below on nDCG@5. Both configurations have an MRR of 1.000, so each placed a relevant project at rank 1 for all 25 queries.

This evaluation therefore does not show hybrid retrieval outperforming dense-only retrieval. Calculated from the per-query scores recorded in the JSON file, the paired standard errors of the differences between the two configurations are 0.012 for F1@5 and 0.018 for nDCG@5. These are estimates of how much the measured differences would vary with a different sample of queries. They do not prove that the two configurations are equivalent, and they do not show that dense-only retrieval is statistically superior; no significance test was performed. The two configurations gave the same F1@5 on 22 of the 25 queries and the same nDCG@5 on 19.

Precision and recall do not trade off in this sweep: both increase together as the dense weight increases. Recall@5 and F1@5 are low in absolute terms because of the number of relevant projects per query. The queries have on average 19.96 relevant projects (between 4 and 59), so five results can contain only a small share of them. The JSON file records that a perfect ranking would reach an F1@5 of only 0.519 under these labels.

The production weights were not changed for this report, and the evaluation alone does not justify changing them. The conclusions apply to this evaluation dataset only. The version 2 relevance labels were produced by a single AI annotator, which also authored projects pm-501 to pm-868, and they were not independently reviewed by a human.

Historical results. The comments in backend/config.py record an earlier nDCG@5 sweep over 25 labelled queries (version 1 labels) and 500 projects: 0.737 for BM25 only, 0.816 at 0.35 / 0.65, 0.845 at 0.50 / 0.50, 0.877 at 0.65 / 0.35, 0.892 at 0.80 / 0.20 and 0.866 for dense only. These historical values were measured on a different catalogue with a different label set. They are not directly comparable with Table 7 or Figures 15 and 16, and they are not included in either.

6.5 Reproducing the Evaluation

With MongoDB running, the catalogue seeded and the index built (Section 5.5), the evaluation is run from the backend directory:

Source: Command line (Windows, from the README)

```
cd backend
venv\Scripts\python -m scripts.seed_db
venv\Scripts\python -m scripts.build_index
venv\Scripts\python -m scripts.evaluate --sweep --verbose
```

The script prints Recall@5, MRR and nDCG@5 for the configured weights, the per-query scores (with --verbose) and the full sweep table.

6.6 Automated Tests

The backend has a pytest suite in backend/tests that covers, among other modules, preprocessing, embeddings, similarity, the evaluation functions, recommendation and the API routes. The frontend has Vitest and React Testing Library tests for its pages and components. The README states 581 backend tests and 37 frontend tests. The tests were not run for this report.

6.7 Limitations

- The evaluation set is small (25 queries) and was labelled within the project.
- The embedding model is used without fine-tuning on the project domain.
- The domain-specific stop-word list was tuned by observation on this catalogue and may need revision as the catalogue grows.
- The vector index must be rebuilt manually after the catalogue changes; /api/health reports when it is out of date.

7. References

1. C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
2. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, Nov. 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.
3. S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019.
4. sentence-transformers, "all-MiniLM-L6-v2," model card, Hugging Face. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (accessed Oct. 14, 2026).
5. "React: Quick start," React documentation. [Online]. Available: https://react.dev/learn (accessed Oct. 14, 2026).
6. Pallets, "Flask documentation (3.1.x)." [Online]. Available: https://flask.palletsprojects.com/ (accessed Oct. 14, 2026).
7. MongoDB, Inc., "MongoDB documentation." [Online]. Available: https://www.mongodb.com/docs/ (accessed Oct. 14, 2026).
8. S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.
9. W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," arXiv:2002.10957, Feb. 2020.
10. D. Brown, "rank_bm25: A collection of BM25 algorithms in Python," GitHub repository. [Online]. Available: https://github.com/dorianbrown/rank_bm25 (accessed Oct. 14, 2026).
11. V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Online, Nov. 2020, pp. 6769–6781, doi: 10.18653/v1/2020.emnlp-main.550.
12. E. M. Voorhees, "The TREC-8 question answering track report," in *Proc. 8th Text REtrieval Conf. (TREC-8)*, NIST Special Publication 500-246, 1999.
13. K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, Oct. 2002, doi: 10.1145/582415.582418.